

Package ‘simulomicsr’

May 4, 2026

Title Pipeline di classificazione LLM per RNAseq da repository pubblici

Version 0.0.0.9006

Description Pipeline R per scaricare metadati di sample RNAseq da GEO/ARCHS4, classificarli via LLM in fatti strutturati e ruoli di design study-level, e produrre confronti per meta-analisi cross-studio.

License file LICENSE

Depends R (>= 4.4)

Imports cli, DBI, digest, fs, glue, httr2, jsonlite, jsonvalidate, purrr, readr, rentrez, rlang, RSQLite, stringr, tibble

Suggests checkmate, covr, devtools, dplyr, ggplot2, here, httptest2, knitr, lintr, qs, readxl, rmarkdown, scales, spelling, tarchetypes, targets, testthat (>= 3.0.0), tidyr, usethis, withr

Config/testthat/edition 3

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.0.0

VignetteBuilder knitr

NeedsCompilation no

Author Corrado Lanera [aut, cre] (ORCID: <<https://orcid.org/0000-0002-0520-7428>>),
Luca Vedovelli [aut] (ORCID: <<https://orcid.org/0000-0003-4847-2333>>)

Maintainer Corrado Lanera <corrado.lanera@ubep.unipd.it>

Contents

simulomicsr-package	3
.anthropic_build_request	3
.anthropic_chat_structured	4
.anthropic_parse_response	4
.load_hgnc	4
.normalize_cell_id	5
.normalize_dose	5

.normalize_duration	6
.openai_build_request	6
.openai_chat_structured	7
.openai_parse_response	7
.stage1_invalid_record	8
.stage2_invalid_record	8
bootstrap_delta_ci	8
build_dev_set	9
build_prompt_stage1	10
build_prompt_stage2	10
cache_init	11
cache_key_for	12
cache_put	12
classify_sample	13
classify_sample_row	13
classify_study	14
compare_with_gold	15
compile_schema	15
design_role_to_binary	16
eval_binary_accuracy	16
eval_per_design_kind	17
fetch_rummageo_signatures	17
fetch_study_summary	18
flag_granularity_disagreement	19
hgnc_dump_path	19
holm_adjust	20
intersect_with_xlsx_and_archs4	20
keyword_design_kind_proxy	21
llm_call_structured	21
load_rummageo_index	22
make_anchor	23
make_inducer_log	23
mcnemar_paired	24
normalize_gene	24
parse_rummageo_labels	25
parse_stage1_response	25
parse_stage2_response	26
read_samples_input	27
read_sample_fixtures_mini	27
reclassify_verbose	28
rummageo_baseline_internal	28
sha256_text	29
stage1_recall_key_fields	29
stage1_schema_validity_rate	30
stratified_sample_gse	30
validate_json	31
wilson_ci	31

simulomicsr-package	<i>simulomicsr: Pipeline di classificazione LLM per RNAseq da repository pubblici</i>
---------------------	---

Description

Pipeline R per scaricare metadati di sample RNAseq da GEO/ARCHS4, classificarli via LLM in fatti strutturati e ruoli di design study-level, e produrre confronti per meta-analisi cross-studio.

Author(s)

Maintainer: Corrado Lanera <corrado.lanera@ubep.unipd.it> ([ORCID](#))

Authors:

- Corrado Lanera <corrado.lanera@ubep.unipd.it> ([ORCID](#))
- Luca Vedovelli <luca.vedovelli@ubep.unipd.it> ([ORCID](#))

`.anthropic_build_request`

Costruisce (senza inviare) la http2 request per Anthropic Messages con structured output via tool_use forzato.

Description

Costruisce (senza inviare) la http2 request per Anthropic Messages con structured output via tool_use forzato.

Usage

```
.anthropic_build_request(  
  model,  
  messages,  
  response_schema,  
  schema_name,  
  max_tokens = 4096L,  
  api_key = NULL  
)
```

```
.anthropic_chat_structured
```

Esegue la chiamata HTTP completa Anthropic e ritorna l'oggetto R parsed.

Description

Chiamato da `llm_call_structured()` quando `provider == "anthropic"`.

Usage

```
.anthropic_chat_structured(
  model,
  messages,
  response_schema,
  schema_name = "response",
  max_tokens = 4096L,
  api_key = NULL,
  ...
)
```

```
.anthropic_parse_response
```

Estrae l'oggetto R dalla risposta Anthropic (tool_use block).

Description

Errori tipizzati:

- `simulomicsr_anthropic_truncated` se `stop_reason` non è `tool_use/end_turn`
- `simulomicsr_anthropic_no_tool_use` se manca il `tool_use` block

Usage

```
.anthropic_parse_response(resp_body)
```

```
.load_hgnc
```

Carica e indicizza il dump HGNC (TSV) in memoria

Description

Carica e indicizza il dump HGNC (TSV) in memoria

Usage

```
.load_hgnc(source_path)
```

<code>.normalize_cell_id</code>	<i>Normalizza un cell identifier per l'anchor</i>
---------------------------------	---

Description

Preferenza: Cellosaurus ID. Fallback: label_raw. Default: "unclear".

Usage

```
.normalize_cell_id(cellosaurus_id, label_raw)
```

Arguments

cellosaurus_id stringa o NULL/NA

label_raw stringa o NULL/NA

Value

stringa canonica

<code>.normalize_dose</code>	<i>Normalizza un valore di dose nella forma canonica dell'anchor</i>
------------------------------	--

Description

Rimuove spazi, normalizza simboli micro, mappa null/NA/"" -> "nodose". Preserva il valore "standard" come placeholder per dosaggi non specificati ma noti dal protocollo.

Usage

```
.normalize_dose(dose)
```

Arguments

dose stringa o NULL/NA

Value

stringa canonica (es. "10nM", "100ng/ml", "nodose", "standard")

<code>.normalize_duration</code>	<i>Normalizza una durata nella forma canonica dell'anchor (ore/giorni)</i>
----------------------------------	--

Description

Converte minuti -> ore (1.5h per 90 min), days -> ore (48h per 2 days) tranne per durate >= 6 giorni dove preserva "Nd" (es. 6d, 14d). Mappa null/NA/"" -> "na".

Usage

```
.normalize_duration(duration)
```

Arguments

<code>duration</code>	stringa o NULL/NA
-----------------------	-------------------

Value

stringa canonica

<code>.openai_build_request</code>	<i>Costruisce (senza inviare) la http2 request per chat/completions con Structured Outputs (strict json_schema).</i>
------------------------------------	--

Description

Costruisce (senza inviare) la http2 request per chat/completions con Structured Outputs (strict json_schema).

Usage

```
.openai_build_request(  
  model,  
  messages,  
  response_schema,  
  schema_name,  
  temperature = NULL,  
  max_tokens = NULL,  
  api_key = NULL  
)
```

`.openai_chat_structured`*Esegue la chiamata HTTP completa e ritorna l'oggetto R parsed.*

Description

Chiamato da `llm_call_structured()` quando `provider == "openai"`.

Usage

```
.openai_chat_structured(  
  model,  
  messages,  
  response_schema,  
  schema_name = "response",  
  temperature = NULL,  
  max_tokens = NULL,  
  api_key = NULL,  
  ...  
)
```

`.openai_parse_response`*Estrae l'oggetto R dalla risposta OpenAI.*

Description

Errori tipizzati:

- `simulomiscr_openai_truncated` se `finish_reason != "stop"`
- `simulomiscr_openai_no_content` se manca `message.content`
- `simulomiscr_openai_bad_json` se `content` non e' JSON parsabile

Usage

```
.openai_parse_response(resp_body)
```

```
.stage1_invalid_record
```

Sample fact "scheletro" usato per registrare un fallimento del classificatore senza rompere la pipeline. Schema-conforme (passa il validator) ma con segnali chiari nei campi extraction e ambiguity_flags.

Description

Sample fact "scheletro" usato per registrare un fallimento del classificatore senza rompere la pipeline. Schema-conforme (passa il validator) ma con segnali chiari nei campi extraction e ambiguity_flags.

Usage

```
.stage1_invalid_record(row, model, reason, detail)
```

```
.stage2_invalid_record
```

Crea un record stage2 invalido per segnalare fallimenti LLM senza interrompere la pipeline

Description

Crea un record stage2 invalido per segnalare fallimenti LLM senza interrompere la pipeline

Usage

```
.stage2_invalid_record(
    series_id,
    reason,
    detail,
    sample_count,
    provider,
    model
)
```

```
bootstrap_delta_ci
```

Bootstrap CI sul delta accuracy tra due classificatori

Description

Resampling sample-level (non bootstrap pairs separati). A ogni iterazione rifa il calcolo accuracy_a - accuracy_b sui sample resampled. Ritorna CI percentile classico.

Usage

```
bootstrap_delta_ci(pred_a, pred_b, n_iter = 1000L, seed = 1812L, conf = 0.95)
```

Arguments

pred_a	character vector "correct"/"wrong"
pred_b	character vector "correct"/"wrong"
n_iter	numero di iterazioni bootstrap (default 1000)
seed	seed per riproducibilit� (default 1812)
conf	livello di confidenza (default 0.95)

Value

list con delta, lower, upper, n_iter

build_dev_set	<i>Costruisce un dev set stratificato per la valutazione iniziale dello Studio 1</i>
---------------	--

Description

Stratificazione (spec §6.1):

- 60% easy_agree: trtctr_EP == trtctr (baseline shallow d'accordo con gold manuale) — controlla regressioni
- 30% disagree_ep_vs_shallow: trtctr_EP != trtctr — qui sta il valore del classificatore LLM
- 10% short_ambiguous: nchar(string) <= 60 — robustezza su input poveri

Usage

```
build_dev_set(samples, n = 100L, seed = 1812L)
```

Arguments

samples	tibble da read_samples_input()
n	dimensione del dev set (default 100; ratio 60/30/10)
seed	seed deterministico (default 1812)

Details

Quando uno strato non ha abbastanza candidati, la funzione fallisce con errore tipizzato (preferiamo failure esplicito a un dev set sbilanciato).

Value

tibble di n sample con colonna aggiuntiva stratum

build_prompt_stage1	<i>Costruisce i messages per la chiamata Stadio 1</i>
---------------------	---

Description

Costruisce i messages per la chiamata Stadio 1

Usage

```
build_prompt_stage1(  
    sample_string,  
    geo_accession,  
    series_id,  
    organism_hint = NULL  
)
```

Arguments

sample_string	testo concatenato dei metadati del sample
geo_accession	GSM id (verra' copiato verbatim nell'output dall'LLM)
series_id	GSE id (idem)
organism_hint	hint opzionale (es. "Homo sapiens"); incluso nello user message solo se non NULL

Value

list di 2 messages (system, user) nel formato OpenAI Chat

build_prompt_stage2	<i>Costruisci il prompt Stadio 2 (study_design) per un GSE</i>
---------------------	--

Description

Crea i messages OpenAI-shape (system + user) e il path allo schema strict. Pronto da passare a `llm_call_structured()`.

Usage

```
build_prompt_stage2(  
    series_id,  
    sample_facts_list,  
    study_summary,  
    model = "openai:gpt-5.5",  
    extra_instruction = NULL  
)
```

Arguments

series_id	GSE accession
sample_facts_list	lista di sample_facts validati (stage1.v3) per tutti i GSM dello studio
study_summary	list con campi series_id/title/summary/overall_design
model	string (es. "openai:gpt-5.5"), inserito nel system per audit
extra_instruction	string opzionale (default NULL) appesa al messaggio user come paragrafo finale. Usata p.es. per investigation verbose-reasoning.

Value

list con campi messages (list di 2 messages role=system/user) e schema_path (path al JSON Schema stage2.v1)

cache_init	<i>Inizializza una cache locale append-only (JSONL + indice SQLite)</i>
------------	---

Description

La cache vive in una directory dir. Per ogni namespace (es. "stage1", "stage2") crea due file: <namespace>.jsonl (record append-only, una riga JSON per put) e <namespace>.sqlite (indice chiave → offset/byte_size dell'ultima versione).

Usage

```
cache_init(dir, namespace = "default")
```

Arguments

dir	directory esistente o creabile dove vivono i file di cache
namespace	nome corto della partizione di cache (es. "stage1")

Details

Idempotente: chiamarla ripetutamente sulla stessa dir non altera lo stato.

Value

oggetto cache (list opaca) usato dalle altre funzioni cache_*

cache_key_for	<i>Costruisce una cache key canonica</i>
	<code><schema_version>:<sha256(payload)></code>

Description

La presenza dello `schema_version` come prefisso garantisce che un bump di schema invalidi automaticamente la cache esistente (vedi spec v5 §5.4).

Usage

```
cache_key_for(schema_version, payload)
```

Arguments

`schema_version` es. "stage1.v3"

`payload` stringa che identifica il contenuto da cacheare

Value

stringa "`<schema_version>:<sha256>`"

cache_put	<i>Inserisce un valore nella cache (append-only su JSONL + indice SQLite)</i>
-----------	---

Description

Note: i valori NA in `value` vengono serializzati come null JSON e tornano come NULL al `cache_get()`.
Se l'identità tipologica di NA è importante per il chiamante, evitare di mettere NA direttamente nei `value` cacheati o convertirli in stringhe come "NA" prima del put.

Usage

```
cache_put(cache, key, value, metadata = list())
```

classify_sample	Classifica un sample RNAseq nello Stadio 1 (sample_facts schema v3)
-----------------	---

Description

Orchestra: build_prompt_stage1 -> llm_call_structured (con cache + Structured Outputs strict) -> parse_stage1_response (enrichment deterministico).

Usage

```
classify_sample(  
  sample_string,  
  geo_accession,  
  series_id,  
  provider = "openai",  
  model = "gpt-5.5",  
  cache = NULL,  
  organism_hint = NULL,  
  ...  
)
```

Arguments

sample_string	stringa di metadati GEO concatenati (input)
geo_accession	GSM id
series_id	GSE id
provider	"openai" (default) o "mock" (test)
model	nome del modello (default "gpt-5.5" – spec sec.5.3.1 dev set)
cache	oggetto cache (cache_init()), oppure NULL per bypass
organism_hint	hint opzionale per lo user message
...	inoltrato all’adapter (es. temperature, max_tokens, .mock_adapter per test)

Value

lista come da llm_call_structured(): value (sample_fact post enrichment), provider, model, validated, cache_hit, raw_response.

classify_sample_row	Wrapper per dynamic branching targets: prende una riga tibble e ritorna direttamente il sample_fact (senza l’envelope di classify_sample()).
---------------------	--

Description

Errori (simulomicsr_schema_error, simulomicsr_openai_*) sono catturati e tradotti in un sample_fact "minimo invalido" con extraction.confidence=0 e ambiguity_flags = ["metadata_inconsistency"] – il filtro di validazione a valle li separerà in sample_facts_invalid.

Usage

```

classify_sample_row(
  row,
  provider = "openai",
  model = "gpt-5.5",
  cache = NULL,
  ...
)

```

classify_study

Classifica il design di uno studio GSE in study_design.stage2.v1

Description

Pipeline: build_prompt_stage2() -> llm_call_structured() -> parse_stage2_response(). Cache trasparente via P1 (la chiave include i messages, separa naturalmente le invocazioni Stadio 1 da Stadio 2). In caso di errore LLM, ritorna un record con .invalid_reason e .invalid_detail (non solleva: il chiamante filtra a valle in study_designs_validated/invalid).

Usage

```

classify_study(
  series_id,
  sample_facts_list,
  study_summary,
  provider = "openai",
  model = "gpt-5.5",
  cache,
  extra_instruction = NULL,
  ...
)

```

Arguments

series_id	GSE accession
sample_facts_list	lista di sample_facts validati (stage1.v3)
study_summary	list con title/summary/overall_design (da fetch_study_summary)
provider	"openai" (default) o futuri provider
model	"gpt-5.5" (default), "gpt-5.4-mini" per batch, ecc.
cache	cache object da cache_init()
extra_instruction	string opzionale (default NULL) appesa al prompt user. Usata da reclassify_verbose() per richiedere chain-of-thought.
...	args passati a llm_call_structured

Value

list (study_design valido stage2.v1) oppure list con campi .invalid_reason/.invalid_detail in caso di failure LLM.

compare_with_gold	<i>Tabella side-by-side gold.xlsx vs simulomicsr P3 vs reclassify verbose</i>
-------------------	---

Description

Tabella side-by-side gold.xlsx vs simulomicsr P3 vs reclassify verbose

Usage

```
compare_with_gold(verbose_design, gold_excel, original_design)
```

Arguments

`verbose_design` design object da reclassify_verbose()
`gold_excel` tibble con colonne geo_accession, series_id, string, gold_binary (treated/control/NA)
`original_design` design object dal run P3 originale

Value

tibble con colonne geo_accession, gold_binary, simulomicsr_p3_role, simulomicsr_reclassify_role, p3_predicted_binary, reclassify_predicted_binary, agreement

compile_schema	<i>Compila uno schema JSON Schema (draft-07) in un validatore richiamabile</i>
----------------	--

Description

Wrapper su `jsonvalidate::json_validator()` con backend Ajv. Il validatore ritornato è una funzione che accetta una stringa JSON e ritorna TRUE/FALSE (con attribute errors se invalido).

Usage

```
compile_schema(schema_path)
```

Arguments

`schema_path` path a un file .json con lo schema

Value

funzione validatrice

design_role_to_binary *Mappa design_role v3 (13 valori) al binary trtctr_predicted*

Description

Implementa la tabella di mapping spec sec.6.2 estesa ai 13 valori del vocabolario design_role. NA = sample escluso dal calcolo accuracy (non un errore, e' una scelta esplicita).

Usage

```
design_role_to_binary(role)
```

Arguments

role character vector di design_role values

Details

Estensione vs spec sec.6.2 originale:

- bystander -> NA (paracrine, non-direttamente perturbed)
- negative_inducer_control -> control (no-Dox arm di sistema inducibile)
- secondary_arm -> treated (trattamento alternativo)

Value

character vector con valori "treated", "control" o NA_character_

eval_binary_accuracy *Calcola binary accuracy per un vettore di predicted vs gold*

Description

Coppie con NA in gold OR predicted vengono escluse dal calcolo. Sensitivity/specificity calcolate considerando "treated" come la classe positiva, "control" come la classe negativa.

Usage

```
eval_binary_accuracy(gold, predicted)
```

Arguments

gold character vector (treated/control/NA)
predicted character vector (treated/control/NA), stessa lunghezza

Value

list con campi n, accuracy, sensitivity, specificity, f1, confusion_matrix

eval_per_design_kind *Breakdown binary accuracy per design_kind*

Description

Breakdown binary accuracy per design_kind

Usage

```
eval_per_design_kind(df)
```

Arguments

df tibble con colonne gold_binary, predicted_binary, design_kind

Value

tibble con una riga per design_kind: design_kind, n, accuracy, sensitivity, specificity, fl

fetch_rummageo_signatures
Recupera le signature group RummaGEO per un GSE con cache filesystem

Description

Interroga il GraphQL di RummaGEO (/graphql) per ottenere i sampleGroups del GSE. Il risultato viene cachato come file JSON in cache_dir per evitare chiamate ripetute. Se il GSE non e' indicizzato in RummaGEO, lancia simulomicsr_rummageo_unavailable.

Usage

```
fetch_rummageo_signatures(  
  gse,  
  cache_dir = NULL,  
  api_base = "https://rummageo.com"  
)
```

Arguments

gse GSE accession (es. "GSE145941")
cache_dir Directory di cache per i risultati JSON (NULL = disattivata)
api_base URL base API RummaGEO (default: "https://rummageo.com")

Details

Formato restituito (identico al formato della cassetta mock):

```
list(
  gse = "GSExxx",
  sampleGroups = list(
    titles = list("1" = "label1", "2" = "label2"),
    samples = list("1" = list("GSMaaa", ...), "2" = list("GSMbbb", ...))
  )
)
```

Value

list con campi gse e sampleGroups

fetch_study_summary	<i>Recupera il summary di uno studio GEO (GSE)</i>
---------------------	--

Description

Wrapper su `rentrez::entrez_summary(db="gds", ...)` con cache filesystem opzionale. Ritorna i campi title, summary, overall_design utilizzati come input dello Stadio 2 (vedi spec sec.4 e sec.5.2).

Usage

```
fetch_study_summary(series_id, cache_dir = NULL)
```

Arguments

series_id	GSE accession (es. "GSE41166"). Validato contro pattern <code>^GSE[0-9]+\$</code> .
cache_dir	Directory di cache (NULL = disattivata). Se esiste un file <code><cache_dir>/<series_id>.json</code> , viene letto senza chiamata di rete.

Details

La cache su disco (un JSON per GSE) e' raccomandata in produzione per evitare hammering di NCBI EUtils (rate limit 3 req/sec senza API key).

`entrez_summary(db = "gds", ...)` non espone `overall_design` nel record summary. Una seconda chiamata `entrez_fetch(db = "gds", rettype = "xml")` sarebbe necessaria per estrarlo. Per ora questo campo viene restituito come `NA_character_` – lo Stadio 2 prompt funziona comunque perche' la maggior parte dell'informazione e' nel campo `summary`.

Value

Lista con campi `series_id`, `title`, `summary`, `overall_design`. `overall_design` puo' essere `NA_character_` se non parsabile dall'XML EUtils (comportamento attuale – vedi dettagli).

flag_granularity_disagreement

Enumera i sample dove simulomicsr ha prodotto un design_role piu' granulare del binary gold.xlsx

Description

Sample con design_role in negative_inducer_control / baseline_t0 / secondary_arm / bystander forniscono informazione che il binary gold non puo' rappresentare. Questi NON sono errori — sono valore aggiunto.

Usage

```
flag_granularity_disagreement(df)
```

Arguments

df tibble con colonne geo_accession, series_id, design_role, gold_binary, predicted_binary

Value

tibble con righe filtrate ai casi granulari, piu' colonna granularity_kind (= design_role)

hgnc_dump_path

Path canonico al dump HGNC nella cache utente

Description

Il dump completo si scarica da <https://www.genenames.org/download/archive/> (nome file: hgnc_complete_set.txt, ~10 MB). In P1 NON scarichiamo automaticamente: l'utente o il futuro plan di setup popola questo path.

Usage

```
hgnc_dump_path()
```

Value

path (non garantito esistente)

holm_adjust	<i>Holm correction per p-values multipli</i>
-------------	--

Description

Wrapper su `p.adjust(method = "holm")` per consistenza nel report. Holm e' uniformly more powerful di Bonferroni a parita' di FWER.

Usage

```
holm_adjust(p_values)
```

Arguments

`p_values` numeric vector di p-values

Value

numeric vector di p-values corretti

intersect_with_xlsx_and_archs4	<i>Intersezione tre-vie GSE: RummaGEO official, xlsx gold, ARCHS4 studies</i>
--------------------------------	---

Description

Produce il pool candidato per la selezione. Ogni GSE deve essere presente in tutti e tre. Se `archs4_studies = NULL`, salta il filtro ARCHS4 (utile per dev offline; la prod imposta ARCHS4 path).

Usage

```
intersect_with_xlsx_and_archs4(rummageo_index, xlsx_df, archs4_studies = NULL)
```

Arguments

`rummageo_index` tibble da `load_rummageo_index()` (col: gse, n_signatures)
`xlsx_df` tibble da `read_samples_input()` (col: geo_accession, series_id, string, trctr_EP)
`archs4_studies` character vector di GSE in ARCHS4 (NULL = no filter)

Value

tibble con colonne `gse`, `n_signatures`, `n_samples_xlsx`, `in_archs4` (TRUE/FALSE/NA), `concat_strings` (concatenazione dei string xlsx per il GSE, usata downstream da `keyword_design_kind_proxy`)

keyword_design_kind_proxy

Inferisce un design_kind candidato da metadata strings via keyword proxy

Description

Applica regex con priorit  fissata (factorial – mediated_effect – treatment_vs_untreated – time_course – knockdown_panel – knockout_vs_wt – disease_vs_normal – treatment_vs_vehicle – unknown). Usato SOLO per stratificazione del pool; la classificazione vera   Stage 2 LLM downstream.

Usage

```
keyword_design_kind_proxy(strings)
```

Arguments

strings	character vector (tipicamente concatenazione dei string xlsx dei sample di un GSE)
---------	--

Value

character vector di design_kind con stessa lunghezza

llm_call_structured *Chiama un LLM con output strutturato validato e cache opzionale*

Description

Punto d'ingresso pubblico del client LLM. Dispatcha sul provider corretto, valida la risposta contro response_schema, e (se cache   fornita) serve dalla cache su hit.

Usage

```
llm_call_structured(
  provider,
  model,
  messages,
  response_schema,
  cache = NULL,
  cache_namespace_version = "v0",
  ...,
  .mock_response = NULL,
  .mock_adapter = NULL
)
```

Arguments

provider	stringa: "openai" o "mock" (per i test). Altri provider in plan futuri.
model	nome del modello (es. "gpt-5.4-mini")
messages	lista di messaggi nello schema OpenAI (role + content)
response_schema	path a un file JSON Schema; la risposta dell'LLM viene validata contro questo schema
cache	oggetto ritornato da cache_init(), o NULL per bypass
cache_namespace_version	stringa che entra nella cache key (es. "stage1.v3"); cambiare questo invalida la cache
...	parametri provider-specifici inoltrati all'adapter
.mock_response	(test only) risposta da iniettare se provider="mock"
.mock_adapter	(test only) function da chiamare al posto dell'adapter reale; ha precedenza su .mock_response

Value

lista con: value (oggetto R parsed dal JSON), provider, model, validated (logico), cache_hit (logico), raw_response (lista grezza).

load_rummageo_index *Scarica indice completo GSE da RummaGEO con cache filesystem*

Description

Pagina via GraphQL gseInfos(first: 1000, offset: ...) finché i nodi ritornati sono < page_size (ultimo batch). Salva il risultato come <cache_dir>/rummageo-index.json. Se il file esiste, lo legge senza rifare il fetch. Splitta GSE multipli (SuperSeries "GSE1,GSE2") in righe separate per atomica di intersezione downstream.

Usage

```
load_rummageo_index(
  cache_dir = NULL,
  api_base = "https://rummageo.com",
  page_size = 1000L,
  max_pages = 50L
)
```

Arguments

cache_dir	directory cache (NULL = no cache, sempre fetch live)
api_base	URL base API (default "https://rummageo.com")
page_size	dimensione pagina (default 1000)
max_pages	limite paginazione difensivo (default 50, ~50k studi)

Value

tibble con colonne gse (atomic GSE accession), n_signatures (somma sample nei sampleGroups)

make_anchor	<i>Costruisci comparability_anchor v3 (13 segmenti) per un sample fact</i>
-------------	--

Description

L'anchor e' una chiave canonica deterministica per cross-studio matching (vedi spec sec.4.3). Selezionata la perturbazione di interesse dal sample, applica le regole R8 (mediated_effect), R9 (variant), R24 (phase), R25 (subcellular default whole_cell), R31 (cell_state default proliferating).

Usage

```
make_anchor(stage1_facts, stage2_role)
```

Arguments

stage1_facts	list (un sample_fact validato stage1.v3)
stage2_role	string: design_role assegnato dallo Stadio 2 al sample (es. "perturbed", "case", "comparison"). Influenza il segmento 12 e, per role case/comparison, kind_effective diventa "disease_vs_normal".

Value

string a 13 segmenti separati da "|"

make_inducer_log	<i>Audit log degli inducenti per perturbazioni mediated_effect (R8)</i>
------------------	---

Description

Per ogni perturbazione con mediated_effect != null, registra l'inducente che make_anchor() perde (es. Dox per sistemi Tet-On). Utile per audit a valle e per QC manuale.

Usage

```
make_inducer_log(stage1_facts)
```

Arguments

stage1_facts	list (un sample_fact validato stage1.v3)
--------------	--

Value

list di entries, una per perturbazione mediated. Vuota se nessuna. Ogni entry ha campi: inducer_kind, inducer_name, mediated_kind, mediated_target.

mcnemar_paired	<i>McNemar test paired su due classificatori</i>
----------------	--

Description

Confronta due classificatori sulle stesse osservazioni rispetto a un gold standard implicito (ognuno e' "correct" o "wrong"). Calcola b (A correct, B wrong) e c (A wrong, B correct), poi statistica $\chi^2 = (b-c)^2 / (b+c)$. Sample con NA in entrambi i predittori esclusi.

Usage

```
mcnemar_paired(pred_a, pred_b, continuity = FALSE)
```

Arguments

pred_a	character vector con valori "correct"/"wrong" del primo classificatore
pred_b	character vector con valori "correct"/"wrong" del secondo classificatore
continuity	applica correzione di continuita' (Yates), default FALSE

Details

Equivalente a `mcnemar.test(table(pred_a, pred_b), correct = continuity)`.

Value

list con statistic, p_value, b, c, n

normalize_gene	<i>Normalizza un nome di gene human a un record canonico HGNC</i>
----------------	---

Description

Strategia di matching (in ordine, primo match vince):

1. symbol esatto case-insensitive
2. alias_symbol esatto case-insensitive
3. prev_symbol esatto case-insensitive

Usage

```
normalize_gene(name, organism = "human", source_path = hgnc_dump_path())
```

Arguments

name	nome di gene da normalizzare (es. "VEGF", "vegfa", "c-Myc")
organism	in P1 solo "human" è supportato
source_path	path al dump HGNC TSV. Default: <code>hgnc_dump_path()</code>

Value

lista con id, preferred_name, resolved_via ("symbol"|"alias_symbol"|"prev_symbol"), oppure NULL se non trovato

parse_rummageo_labels *Estrae assegnazione control/treated per GSM dai sampleGroups RummaGEO*

Description

Converte il campo sampleGroups (formato gseInfos di RummaGEO) in un tibble GSM -> label. La convenzione e': il gruppo con indice numerico piu' basso e' classificato "treated" (tipicamente il braccio perturbato), il gruppo con indice piu' alto e' "control" (basale/controllo). Questo e' consistente con i pattern osservati nell'API reale (2026-05-02).

Usage

```
parse_rummageo_labels(data)
```

Arguments

data list ritornato da fetch_rummageo_signatures()

Details

Se data\$sampleGroups e' vuoto o assente, ritorna un tibble vuoto.

Value

tibble con colonne geo_accession e rummageo_label ("treated" o "control")

parse_stage1_response *Enrichment deterministico della risposta LLM Stadio 1*

Description

Forza i campi che NON devono dipendere dall'LLM:

- geo_accession e series_id ricopiati dall'input (anti-allucinazione)
- extraction.schema_version = "stage1.v3"
- extraction.model settato dal caller R (non dal contenuto LLM)
- extraction.raw_input_hash calcolato deterministicamente da sha256(sample_string)

Usage

```
parse_stage1_response(raw, sample_string, geo_accession, series_id, model)
```

Arguments

raw	lista R parsed dal JSON di risposta LLM
sample_string	testo originale del sample
geo_accession	GSM id originale
series_id	GSE id originale
model	nome del modello (es. "gpt-5.5"); verra' prefissato con "openai:" nell'output

Details

Tutti gli altri campi sono lasciati al modello (Structured Outputs ne ha gia' verificato la conformita' allo schema).

Value

lista R con i campi forzati

parse_stage2_response	<i>Enrichi la risposta stage2 con metadata deterministici dal chiamante</i>
-----------------------	---

Description

Forza series_id, schema_version, model e input_sample_count dal contesto del caller. Non siamo mai completamente fiduciosi che l'LLM abbia interpretato correttamente questi campi.

Usage

parse_stage2_response(raw, series_id, sample_count, model)

Arguments

raw	Risposta parsed JSON (list), gia' validata contro stage2.v1
series_id	GSE accession (forzato)
sample_count	numero intero di sample input (forzato come input_sample_count)
model	string modello usato (forzato come model)

Value

raw (list) con series_id, extraction\$schema_version='stage2.v1', extraction\$model, extraction\$input_samp sovrascritti. Se extraction non esiste, viene creato come lista vuota.

read_samples_input	<i>Legge il xlsx classificato in un tibble normalizzato</i>
--------------------	---

Description

Le colonne attese sono quelle documentate in data-raw/README.md. Tutto viene letto come character (stabile), il caller converte se serve.

Usage

```
read_samples_input(path, n_max = Inf)
```

Arguments

path	path al file xlsx
n_max	numero massimo di righe da leggere (default Inf)

Value

tibble con colonne geo_accession, series_id, string, trtctr_EP, trtctr, treat, gold

read_sample_fixtures_mini	<i>Legge il TSV di fixture sample mini bundled nel pacchetto</i>
---------------------------	--

Description

Per test e vignette: 8 sample stratificati estratti dal xlsx (script in data-raw/build-sample-fixtures-mini.R).

Usage

```
read_sample_fixtures_mini()
```

Value

tibble con colonne geo_accession, series_id, string, trtctr_EP, trtctr, treat, gold, stratum

reclassify_verbose	<i>Re-classify uno studio con prompt verbose-reasoning</i>
--------------------	--

Description

Chiama `classify_study()` con un'istruzione aggiuntiva (`extra_instruction`) che chiede esplicitamente al modello di esplicitare la chain-of-thought per (a) assegnazione GSM -> `replicate_group`, (b) identificazione control/ treatment arm, (c) inferenza `design_kind`. Il reasoning risultante e' inserito nel campo `.reasoning` del design object.

Usage

```
reclassify_verbose(  
  series_id,  
  sample_facts_list,  
  study_summary,  
  cache = NULL,  
  provider = "openai",  
  model = "gpt-5.5"  
)
```

Arguments

<code>series_id</code>	GSE accession
<code>sample_facts_list</code>	list di <code>sample_facts</code> validati per il GSE
<code>study_summary</code>	list ritornato da <code>fetch_study_summary()</code>
<code>cache</code>	<code>cache_init()</code> per Stadio 2 (se NULL nessuna cache)
<code>provider</code>	passato a <code>classify_study()</code> (default "openai")
<code>model</code>	passato a <code>classify_study()</code> (default "gpt-5.5")

Details

Usato per investigation GSE145941 (P3.5-A sec. 5).

Value

design object con campo aggiuntivo `.reasoning`

rummageo_baseline_internal	<i>Replica fallback dell'algoritmo RummaGEO (Marino 2024)</i>
----------------------------	---

Description

Quando l'API ufficiale non e' disponibile (es. GSE non indicizzato in RummaGEO), applichiamo internamente lo stesso algoritmo di base: keyword matching su control terms + default "treated" per il resto. Questo NON e' equivalente al benchmark "vs RummaGEO ufficiale" — controlliamo noi il metodo. Documentato come baseline interno nel report.

Usage

rummageo_baseline_internal(samples)

Arguments

samples tibble con colonne geo_accession, string

Details

Differenza vs Marino 2024: NON facciamo K-means (non ha senso su subset di 15 GSE; Marino lo usa per gestire studi grandi). Solo keyword matching word-boundary per evitare match parziali.

Value

tibble con colonne geo_accession, rummageo_label (treated/control)

sha256_text	<i>Calcola lo SHA-256 di una stringa testuale UTF-8</i>
-------------	---

Description

Calcola lo SHA-256 di una stringa testuale UTF-8

Usage

sha256_text(x)

Arguments

x stringa di lunghezza 1

Value

stringa esadecimale di 64 caratteri

stage1_recall_key_fields	<i>Recall di campi chiave nei sample_fact validati</i>
--------------------------	--

Description

- "with_perturbation" = perturbations[1].kind non in {none, unclear, NA}
- "with_cell_type" = cell_context.cell_type_or_line_raw non NULL

Usage

stage1_recall_key_fields(facts_validated)

Arguments

facts_validated
lista di sample_fact validati

Value

```
list(n_samples, n_with_perturbation, n_with_cell_type, recall_perturbation, recall_cell_type)
```

stage1_schema_validity_rate

Metriche di base dello Stadio 1 sul dev set

Description

Tutte le funzioni accettano una lista di `sample_fact` (oggetti R parsed dal JSON) e ritornano una lista flat di numeri/conteggi adatta a essere impacchettata in tibble per il report. Rate di `sample_facts` che superano la validazione schema

Usage

```
stage1_schema_validity_rate(facts_validated, facts_invalid)
```

Arguments

```
facts_validated      lista di sample_fact validati
facts_invalid        lista di sample_fact invalidati (post catch in classify_sample_row)
```

Value

```
list(n_validated, n_invalid, n_total, validity_rate)
```

stratified_sample_gse *Stratified sampling deterministico dal pool candidato*

Description

Per ogni categoria k in target, prende min(target[k], n_disponibili) GSE (random uniform su quelli con design_kind_proxy == k). Categoria povera (n_disponibili < target) -> prendi tutti, deficit ridistribuito su treatment_vs_vehicle. GSE con design_kind_proxy = "unknown" sono esclusi dalla selezione (non aggiungono diversita' nota).

Usage

```
stratified_sample_gse(pool, target, seed = 1812L)
```

Arguments

pool	tibble con colonne gse, design_kind_proxy, n_signatures, n_samples_xlsx, in_archs4 (opzionale)
target	named integer vector (es. c(factorial = 15, time_course = 15, ...)); somma = N totale desiderato
seed	seed RNG (default 1812)

Details

Determinismo garantito da seed esplicito (default 1812 per consistenza con P2/P3).

Value

tibble con stessi campi di pool filtrato ai GSE selezionati

validate_json	<i>Valida un oggetto R o una stringa JSON contro un validatore compilato</i>
---------------	--

Description

Valida un oggetto R o una stringa JSON contro un validatore compilato

Usage

```
validate_json(x, validator)
```

Arguments

x	lista R (verrà serializzata) oppure stringa JSON
validator	funzione ritornata da compile_schema()

Value

lista con valid (logico) e errors (character vector, vuoto se valid)

wilson_ci	<i>Wilson 95% CI per proporzione</i>
-----------	--------------------------------------

Description

Implementa la formula di Wilson (1927) per intervallo di confidenza su proporzione binomiale. Preferito a normal approximation per p vicino ai bordi (0 o 1) o n piccolo. Ritorna NA se n=0.

Usage

```
wilson_ci(successes, n, conf = 0.95)
```

Arguments

successes	numero di successi (intero)
n	numero totale di osservazioni (intero)
conf	livello di confidenza (default 0.95)

Value

list con campi estimate, lower, upper, n